

CS240: Lab 10

TAs: Anvay

March 30, 2026

Objective

In this **ungraded outlab**, you are tasked with implementing **tabular Q-learning** to learn a policy for an agent that is navigating a stochastic gridworld environment.

Environment: Frozen Lake (5×5)

You are given a 5×5 gridworld:

S	F	F	F	F
F	H	F	F	H
F	F	F	F	H
F	F	H	F	F
H	F	F	F	G

States

There are 4 types of states: **S (blue)**: Start state (0,0), **G (green)**: Goal (terminal, reward = 1), **H (red)**: Hole (terminal, reward = 0), **F (light blue)**: Frozen (safe state). Each state is a grid coordinate: $s = (i, j)$, $i, j \in \{0, 1, 2, 3, 4\}$.

Actions

The actions that the agent may take are: $\{0 = \text{Up}, 1 = \text{Down}, 2 = \text{Left}, 3 = \text{Right}\}$.

Rewards

The agent receives a +1 reward for reaching the Goal state, and 0 otherwise (for all states, including holes).

Transitions

The environment is **slippery**. When the agent takes an action to move in a certain direction, the environment will generate the next state stochastically according to the distributions given below. If a move would leave the grid, the agent stays in the same state.

Action Taken	Actual Movement (with probabilities)
$\uparrow$ Up (0)	$\uparrow$ Up (0.75), $\leftarrow$ Left (0.125), $\rightarrow$ Right (0.125)
$\downarrow$ Down (1)	$\downarrow$ Down (0.40), $\leftarrow$ Left (0.30), $\rightarrow$ Right (0.30)
$\leftarrow$ Left (2)	$\leftarrow$ Left (0.40), $\uparrow$ Up (0.30), $\downarrow$ Down (0.30)
$\rightarrow$ Right (3)	$\rightarrow$ Right (0.40), $\uparrow$ Up (0.30), $\downarrow$ Down (0.30)

An episode ends when either the agent reaches the Goal, falls into a Hole, or the number of steps reaches 200.

Q-Learning

Tabular Q-Learning is a popular model-free reinforcement learning algorithm used to find the optimal policy in a Markov Decision Process (MDP). It is highly memory-efficient, requiring only $\mathcal{O}(|S||A|)$ space compared to the $\mathcal{O}(|S|^2|A|)$ space required by model-based approaches. Consider an agent at state s^t , interacting with the environment by taking an action a^t sampled from its current policy π^t . Upon transitioning, the agent receives a reward r^t and observes the next state s^{t+1} .

You will implement the Q-learning update rule:

$$Q^{t+1}(s^t, a^t) \leftarrow Q^t(s^t, a^t) + \alpha \left[r^t + \gamma \max_{a \in \mathcal{A}} Q^t(s^{t+1}, a) - Q^t(s^t, a^t) \right].$$

where $\alpha \in (0, 1]$ is the learning rate and $\gamma \in [0, 1)$ is the discount factor.

Crucially, Q-learning is an **off-policy** algorithm. This means it evaluates and improves the optimal policy independently of the agent's current policy π^t . It achieves this by always evaluating the target value using the absolute maximum Q-value of the next state, $\max_a Q^t(s^{t+1}, a)$.

As a contrasting example, **SARSA** (State-Action-Reward-State-Action) is an **on-policy** algorithm. Instead of assuming the best possible action, it updates its estimates based explicitly on the actual action a^{t+1} recommended by the policy π^t in the next state:

$$Q^{t+1}(s^t, a^t) \leftarrow Q^t(s^t, a^t) + \alpha [r^t + \gamma Q^t(s^{t+1}, a^{t+1}) - Q^t(s^t, a^t)].$$

While the core focus of this practical lab is understanding Q-learning, this mathematical update contrast highlights fundamental differences in their theoretical convergence guarantees.

Convergence to the Optimal Policy (Q^*)

Both Q-learning and SARSA require that all state-action pairs be visited infinitely often, and that the learning rate α satisfies the standard Robbins-Monro conditions [1] ($\sum_{t=0}^{\infty} \alpha_t = \infty$ and $\sum_{t=0}^{\infty} \alpha_t^2 < \infty$). However, their convergence behaviors and exploration rules fundamentally differ:

- **Q-learning (Off-Policy):** Converges directly to the optimal action-value function, Q^* . It can use *any* generic exploration policy (like a fixed ϵ -greedy framework) as long as complete state-action coverage is maintained.
- **SARSA (On-Policy):** Converges to the action-value function of the current behavior policy, Q^π . To successfully converge to the absolute optimal Q^* , its exploration must strictly satisfy **GLIE** (Greedy in the Limit with Infinite Exploration), meaning the probability of exploratory moves must decay to zero over time (e.g., $\epsilon_t = 1/t$).

Task 1: Implementation

Complete the function `q_learning(env, max_steps)` in the script `QLearning.py`. Your implementation should learn a tabular Q-function through iterative interaction with the environment. You must utilize a formalized exploration strategy, such as ϵ -greedy action selection. Return a **deterministic greedy policy** derived from your final Q-table as a 5×5 array, where each entry is an explicit action in $\{0, 1, 2, 3\}$. Note that terminal states (H, G) can technically contain any action value as they do not permit subsequent transitions.

Implementation Details and Tips

The evaluation code saves your policy to a CSV file and tests its success rate over multiple episodes using only greedy actions.

To find the best policy, you need to tune several hyperparameters: your Q-table initialization, the number of training episodes, the learning rate (α), the discount factor (γ), and your exploration strategy. We highly recommend using an ϵ -greedy approach and slowly decreasing ϵ over time. This helps your agent transition smoothly from exploring the grid to using known safe paths.

Don't be discouraged if early learning is slow. Because rewards in this environment are sparse, it can easily take tens of thousands of training episodes before the agent consistently reaches the goal.

Task 2: Experiments

(a) Hyperparameter Tuning and Epsilon Decay

Your agent's performance heavily depends on its hyperparameters. Since this lab is ungraded, we strongly encourage you to experiment with them to see their impact.

- **Discount factor (γ):** Compare a lower value (e.g., $\gamma = 0.9$) to a high one ($\gamma = 0.999$) to see which is better at finding a safe path.
- **Exploration strategy:** Compare a fast epsilon decay (e.g., $\epsilon \leftarrow \epsilon \times 0.99$) against a slow one (e.g., $\epsilon \leftarrow \epsilon \times 0.99995$). Observe how the decay rate affects the agent's final success rate and convergence.

(b) Reward Shaping: The Cost of Time

By default, the agent is not penalized for wandering safely on the ice. Modify your training loop to include a **small negative step penalty** (e.g., $r_{\text{step}} = -0.01$ or -0.05) for every step that does not reach the goal.

Retrain your agent and compare this new policy against the baseline (which has no step penalty). During evaluation, track the average number of steps it takes to reach the goal in successful episodes.

Finally, reflect on the following:

- Does the step penalty make the agent reach the goal faster?
- How does this penalty affect the agent's overall safety (comparing the success rate against the rate of falling into holes)?

Expected Outcome

Your baseline algorithm implementation should be fully capable of reliably reaching the goal with a **success rate exceeding 90%, while completely avoiding all holes (0% failure rate)**. You are highly encouraged to visualize your derived policy using the provided supplementary script `VisualizeAgent.py`.

References

- [1] Herbert E. Robbins. A stochastic approximation method. *Annals of Mathematical Statistics*, 22:400–407, 1951.